Argo CD ⭐ ⭐ — Interview-Ready Q&A

1. What is Argo CD?
Argo CD is a **GitOps-based continuous delivery tool for Kubernetes**.
It continuously monitors the Kubernetes manifests/configuration stored in Git and makes sure the Kubernetes cluster matches the desired state defined in Git.
Simple ga gurthupettuko:
Git = **Desired State**
Argo CD = **Git state ni Kubernetes lo maintain chestundi**
Eng: Argo CD is a declarative GitOps continuous delivery tool used to manage and deploy applications to Kubernetes.

---

2. What is GitOps?
GitOps is an approach where **Git is treated as the source of truth** for infrastructure or application deployment configuration. Instead of manually changing Kubernetes resources, we make changes in Git. A GitOps tool such as Argo CD detects the change and synchronizes the cluster.
Developer
↓
Git
↓
Argo CD
↓
Kubernetes
Simple ga gurthupettuko:
Git lo change → Argo CD → Kubernetes update
Eng: GitOps uses Git as the source of truth and automatically reconciles the target environment with the configuration stored in Git.

---

3. Why do we use Argo CD?
Without Argo CD, deployments can involve manually running kubectl commands or having CI pipelines directly modify the Kubernetes cluster.
Argo CD provides:

- Git-based deployment
- Automatic synchronization
- Drift detection
- Application health monitoring
- Easy rollback
- Deployment history
- Kubernetes-native management

Simple ga gurthupettuko:
Argo CD = **Automated Kubernetes Deployment + GitOps**
Eng: Argo CD simplifies Kubernetes deployments by continuously reconciling the cluster with the desired configuration stored in Git.

---

4. What is an Argo CD Application?
An Argo CD **Application** represents a deployed application and defines important information such as:

- Git repository
- Path
- Target revision
- Kubernetes destination
- Sync configuration

For example:
Application: payment-app
Git Repo: github.com/company/k8s-manifests
Path: payment/prod
Cluster: production
Namespace: payment
Simple ga gurthupettuko:
Application = **Git source + Kubernetes destination**
Eng: An Argo CD Application is a Kubernetes resource that defines the desired source configuration and destination where the application should be deployed.

---

5. What is an Argo CD Project?
An Argo CD **Project** provides logical grouping and access restrictions for Applications.
It can control:
1. Which repositories can be used 2,Which clusters can be deployed to 3, Which namespaces can be used
4 ,Which resource types are allowed
Simple ga gurthupettuko: Project = **Application ki boundaries/security rules**
Eng: An Argo CD Project provides logical organization and security boundaries for applications, repositories, destinations, and resource types.

6. What is a Repository in Argo CD?
A Repository is the Git source from which Argo CD retrieves application manifests.
For example:
Git Repository
├── dev
├── qa
└── prod
Argo CD reads the required path and compares the desired configuration with the live Kubernetes state.
Simple ga gurthupettuko:
Repository = **Argo CD manifests teesukune Git place**
Eng: An Argo CD repository is a configured Git source containing the declarative application manifests or configuration used for deployment.

---

7. What is Sync in Argo CD?
Sync means making the live Kubernetes environment match the desired state stored in Git.
For example, Git contains:
replicas: 5
but Kubernetes currently has:
replicas: 3
When Argo CD synchronizes, it applies the desired configuration so the cluster reaches the Git-defined state.
Simple ga gurthupettuko:
Git = 5 replicas
Cluster = 3 replicas
Sync → Cluster = **5 replicas**
Eng: Sync reconciles the Kubernetes live state with the desired state defined in the configured Git repository.

---

8. What is Manual Sync?
In Manual Sync, Argo CD detects that the Git state and cluster state differ, but the deployment is not automatically applied.
An engineer can review the change and trigger synchronization manually.
Git Change
↓
Argo CD detects difference
↓
Engineer reviews
↓
Manual Sync
↓
Kubernetes
Simple ga gurthupettuko:
Manual Sync = **Engineer approval/action tarvata deployment**
Eng: Manual Sync requires an operator to initiate synchronization after reviewing the detected changes.

---

9. What is Auto Sync?
Auto Sync allows Argo CD to automatically synchronize changes from Git to Kubernetes.
For example:
Developer
↓
Git Push
↓
Argo CD detects change
↓
Auto Sync
↓
Kubernetes Deployment
This reduces the need for manual deployment actions.
Simple ga gurthupettuko:
Git Push → Argo CD → **Automatic Deployment**
Eng: Auto Sync enables Argo CD to automatically apply detected desired-state changes from Git to the Kubernetes cluster.

---

10. Difference between Manual Sync and Auto Sync?
Manual Sync:
Git Change → Argo CD → Engineer → Sync → Kubernetes
Auto Sync:
Git Change → Argo CD → Sync → Kubernetes
Manual Sync provides more human control, while Auto Sync provides more deployment automation.
Simple ga gurthupettuko:
Manual = **Human trigger**
Auto = **Argo trigger**
Eng: Manual Sync requires an operator to initiate deployment, while Auto Sync allows Argo CD to synchronize changes automatically.

---

11. What is Health Status in Argo CD?
Health Status indicates the operational health of the Kubernetes resources managed by an Argo CD Application.
For example, an Application can show states such as:
Healthy
Progressing
Degraded
Suspended
Missing
Unknown
If a Deployment is healthy and its Pods are running correctly, the Application may be shown as Healthy.
Simple ga gurthupettuko:
Health = **Application currently okay ga run avtunda?**
Eng: Argo CD Health Status provides an assessment of the operational state of resources managed by an Application.

---

12. What is Sync Status?
Sync Status tells us whether the Kubernetes live state matches the desired state in Git.
Commonly:
Synced
OutOfSync
Unknown
For example:
Git replicas = 5
Kubernetes replicas = 3
Argo CD can identify this difference as **OutOfSync**.
Simple ga gurthupettuko:
Sync Status = **Git state and Cluster state same aa?**
Eng: Sync Status indicates whether the live Kubernetes resources match the desired configuration stored in Git.

---

13. What is the difference between Sync Status and Health Status?
These two are very important in interviews.
Sync Status asks:
Does the cluster match Git?
Health Status asks:
Is the deployed application/resource healthy?
Example:
Sync Status = Synced
Health Status = Degraded
This means the cluster matches Git, but the application itself has a problem.
Simple ga gurthupettuko:
Sync = **Configuration correct aa?**
Health = **Application healthy aa?**
Eng: Sync Status represents desired-state alignment with Git, while Health Status represents the operational health of the deployed resources.

---

14. What is Drift in GitOps?
Drift occurs when the actual Kubernetes state differs from the desired state stored in Git.
For example, Git says:
replicas: 3
Someone manually changes Kubernetes:
kubectl scale deployment app --replicas=5
Now:
Git = 3
Cluster = 5
Argo CD detects this difference as drift/OutOfSync.
Simple ga gurthupettuko:
Git ≠ Cluster → **Drift**
Eng: Drift occurs when the live Kubernetes configuration differs from the declarative desired state stored in Git.

---

15. What happens if someone manually changes a Kubernetes resource?
Suppose Git contains:
replicas: 3
but someone manually changes the Deployment to 5 replicas.
Argo CD detects that the live state differs from Git and marks the Application as **OutOfSync**.
If Auto Sync and the relevant reconciliation behavior are configured, Argo CD can bring the resource back toward the Git-defined desired state.
Simple ga gurthupettuko:
Manual Change → Git ≠ Cluster → **OutOfSync**
Eng: If a resource is manually modified outside Git, Argo CD can detect the drift and, when configured, reconcile the resource back to the desired Git state.

16. How do you rollback a deployment using Argo CD?
Argo CD maintains application synchronization history.
If a deployment causes an issue, we can select a previous known-good revision and synchronize the application back to that desired version.
Typical flow:
Current Version
↓
Issue
↓
Previous Git Revision
↓
Argo CD Sync
↓
Previous Version
Simple ga gurthupettuko:
Bad Version → Previous Git Version → **Sync → Rollback**
Eng: I can roll back an Argo CD application by reverting to a previous known-good revision and synchronizing that desired state to Kubernetes.

---

17. Why is Git history important in Argo CD?
Git history provides a record of configuration changes.
We can identify:

- Who changed the configuration
- What was changed
- Which commit introduced the change
- Which version should be restored

This makes deployments more traceable and rollback-friendly.
Simple ga gurthupettuko:
Git = **Deployment history + Audit trail + Rollback source**
Eng: Git history provides traceability for deployment configuration changes and helps identify and restore previous known-good versions.

---

18. What is the role of Argo CD in a CI/CD pipeline?
Argo CD is commonly used for the **CD/deployment part** of the pipeline.
A typical flow:
Developer
↓
Git Application Code
↓
CI Pipeline
↓
Build + Test
↓
Docker Image
↓
Container Registry
↓
Update Kubernetes Manifest
↓
GitOps Repository
↓
Argo CD
↓
Kubernetes
Simple ga gurthupettuko:
CI = **Build/Test**
Argo CD = **Kubernetes Deployment**
Eng: In a GitOps pipeline, CI builds and publishes the application artifact, while Argo CD continuously reconciles the Kubernetes environment from the deployment configuration stored in Git.

---

19. Does Argo CD build Docker images?
No.
Argo CD is primarily a **Kubernetes continuous delivery and GitOps tool**. It does not normally build the application Docker image.
A CI tool performs the build:
CI
↓
Docker Build
↓
Registry
Then the deployment configuration references the new image version, and Argo CD deploys it.
Simple ga gurthupettuko:
CI = **Build Image**
Argo CD = **Deploy Image**
Eng: Argo CD does not normally build application images; CI systems build and publish images, while Argo CD deploys the desired image configuration to Kubernetes.

---

20. Explain a real-time Argo CD deployment flow.
Suppose a developer changes application code.
The CI pipeline builds a new Docker image and pushes it to the container registry. The deployment repository is then updated with the new image tag.
Argo CD monitors the Git repository, detects the change, and identifies the application as OutOfSync. If Auto Sync is enabled, Argo CD applies the new Kubernetes manifests.
Kubernetes creates or updates the required Pods, and Argo CD monitors their health.
Developer
↓
Git
↓
CI Pipeline
↓
Docker Build
↓
Container Registry
↓
GitOps Manifest Update
↓
Argo CD
↓
Sync
↓
Kubernetes
↓
Pods
↓
Health Check
↓
Healthy
Simple ga gurthupettuko:
Code → CI → Image → GitOps Repo → Argo CD → Kubernetes → Healthy
Eng: In a typical GitOps deployment, CI builds and publishes the image, the desired deployment configuration is updated in Git, and Argo CD detects and synchronizes that desired state into Kubernetes while monitoring application health.

---

21. Interview Scenario: Argo CD shows OutOfSync. What will you check?
First, I compare the desired Git configuration with the live Kubernetes resource.
Then I check:
Git Manifest
↓
Argo CD Diff
↓
Live Kubernetes Resource
↓
Manual Changes?
↓
Sync Configuration?
I also check whether the difference is expected or caused by an external controller modifying the resource.
Simple ga gurthupettuko:
OutOfSync → **Diff chudu → Expected change aa? → Manual change aa? → Reconcile**
Eng: For an OutOfSync application, I inspect the Git-to-cluster diff, determine whether the difference is intentional, and then synchronize or correct the source configuration as appropriate.

---

22. Interview Scenario: Argo CD Application is Synced but Health is Degraded. What does it mean?
It means the Kubernetes resources match the desired Git configuration, but the application is not operating correctly.
For example:
Sync Status = Synced
Health Status = Degraded
I would then troubleshoot the Kubernetes workload using:
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
I would investigate crashes, failed probes, insufficient resources, configuration issues, or dependency failures.
Simple ga gurthupettuko:
Synced = **Configuration correct**
Degraded = **Application problem**
Eng: A Synced but Degraded application means the cluster matches Git, but one or more deployed resources are not operating in a healthy state.

---

23. Interview Scenario: Auto Sync is enabled but deployment is not happening. What will you check?
First, I verify that Argo CD actually detected the Git change.
Then I check:

- Application Sync Status
- Git repository connectivity
- Correct branch/revision
- Correct manifest path
- Application permissions
- Sync configuration
- Kubernetes API access
- Argo CD application/controller logs

Simple ga gurthupettuko:
Git Change → Detect → Sync → Kubernetes
Any step fail ayindo identify chestha.
Eng: If Auto Sync is not deploying changes, I verify Git detection, repository and revision configuration, application permissions, sync settings, Kubernetes access, and Argo CD logs.

---

24. Interview Scenario: How would you safely deploy a new version using Argo CD?
First, I make the deployment change in Git rather than directly modifying Kubernetes.
Then I verify the manifest and image tag through CI/testing processes.
After the Git change is merged, Argo CD detects the desired-state change. Depending on the environment, I can use Manual Sync for controlled Production deployment or Auto Sync for automated environments.
After deployment, I verify:
Sync Status
Health Status
Pod Status
Application Logs
Metrics
Simple ga gurthupettuko:
Git Change → Review → Argo CD → Deploy → **Health Verify**
Eng: I use Git as the source of truth, validate the change before deployment, synchronize through Argo CD, and verify application health after deployment.

---

25. What are the main advantages of Argo CD?
The major advantages are:

- Git as the source of truth
- Declarative deployments
- Automatic synchronization
- Drift detection
- Deployment history
- Easy rollback
- Kubernetes-native approach
- Application health visibility
- Better deployment consistency

Simple ga gurthupettuko:
Argo CD = **GitOps + Automation + Drift Detection + Visibility + Rollback**
Eng: Argo CD provides declarative GitOps-based Kubernetes delivery with automated reconciliation, drift detection, health monitoring, and reliable deployment management.